

Patient Record Data Pipeline

PatientRecordPipeline01

End-to-End Azure Data Engineering Portfolio Project

Healthcare Analytics | Medallion Architecture | Delta Lake | Unity Catalog

Component	Technology	Purpose
Ingestion	Azure Data Factory	CSV to Parquet copy activity into ADLS Bronze layer
Storage	Azure Data Lake Storage Gen2	Medallion architecture: raw / bronze / silver / gold
Orchestration	ADF Pipeline + Schedule Trigger	Daily automated pipeline run
Minor Transform	Azure Databricks (MinorT)	Data cleaning and Silver Delta table creation
Major Transform	Azure Databricks (MajorT)	Delta MERGE to Gold + JDBC load to Azure SQL
Secret Management	Azure Key Vault	Databricks token and SQL credentials
Data Governance	Databricks Unity Catalog	Schema management across silver and gold layers
Reporting	Power BI	Interactive healthcare analytics dashboards

1. Project Overview

This project implements a production-grade, automated data pipeline on Microsoft Azure that ingests raw hospital patient records, applies comprehensive data quality transformations, and exposes clean, analytics-ready data through a Power BI reporting layer.

The pipeline follows the Medallion Architecture pattern (Raw → Bronze → Silver → Gold), a widely adopted industry standard for building reliable and scalable data lakehouses. It demonstrates real-world data engineering skills including schema enforcement, large-scale PySpark data cleansing, ACID-compliant upserts using Delta Lake, secret management with Azure Key Vault, and Unity Catalog data governance.

1.1 Problem Statement

Hospital patient records arrive as raw CSV files containing significant data quality issues: inconsistent patient ID formats, mixed-case names, multiple date formats, sentinel null values (-999, N/A), negative length-of-stay values caused by erroneous discharge dates, malformed emails, duplicate records, and embedded unit strings in numeric fields. This pipeline ingests, validates, cleanses, and standardises this data for downstream clinical and operational reporting.

1.2 Key Outcomes

- 13+ distinct data quality rules applied across patient demographics, vitals, and financial fields
- Fully automated daily pipeline triggered on a schedule
- Delta MERGE pattern ensures idempotent, upsert-safe loads to the Gold layer
- Secrets managed via Azure Key Vault - zero plaintext credentials in code or config files
- Power BI report connected to Azure SQL serving layer for interactive dashboards

2. Architecture

2.1 High-Level Data Flow

The pipeline follows a strict left-to-right data flow across five layers:

Layer	Storage Location	Format	Description
Raw / Source	ADLS: raw container	CSV	Source file landing zone, preserved unchanged
Bronze	ADLS: patient-record/bronze/	Parquet (Snappy)	ADF Copy Activity converts CSV to Parquet
Silver	Unity Catalog: silver schema	Delta Lake	Cleaned, typed, deduplicated patient records
Gold	Unity Catalog: gold schema	Delta Lake	MERGE-based upsert; analytics-ready dataset
Serving	Azure SQL DB staging table	Azure SQL	Loaded via JDBC for Power BI consumption

2.2 ADF Pipeline Activity Chain

Azure Data Factory orchestrates the end-to-end pipeline through three sequential activities, each dependent on the success of the previous:

Step	Activity Name	Type	Dependency
1	Data2Bronze	Copy Activity (CSV to Parquet)	None - pipeline entry point
2	MinorTrans	Databricks Notebook: MinorT	Data2Bronze must succeed
3	MajorTrans	Databricks Notebook: MajorT	MinorTrans must succeed

Each activity is configured with a 12-hour timeout, 0 retries, and a 30-second retry interval. The pipeline is triggered by a Schedule Trigger (PatientData_Merge) set to run daily, ensuring fresh data is available for reporting each morning.

2.3 Azure Services Used

Service	Role in Pipeline
Azure Data Lake Storage Gen2	Central data lake hosting all medallion layers
Azure Data Factory	Orchestration engine: ingestion, sequencing, scheduling
Azure Databricks	PySpark transformation notebooks (MinorT and MajorT)
Azure Key Vault	Secure storage of Databricks token and SQL credentials
Databricks Unity Catalog	Governed schema and table management for silver and gold
Azure SQL Database	Serving layer consumed by Power BI via JDBC
Power BI	Interactive dashboards for healthcare analytics

3. Data Pipeline Components

3.1 Linked Services

Linked Service	Type	Purpose	Auth Method
Is_pRecord_csv	AzureBlobFS	Connect to ADLS raw container (CSV source)	Encrypted credential
Is_pRecord_parq	AzureBlobFS	Connect to ADLS bronze container (Parquet sink)	Encrypted credential
Is_Minor	AzureDatabricks	Connect to Databricks workspace (both notebooks)	Key Vault secret token
PatientKV	AzureKeyVault	Connect to Azure Key Vault for secret retrieval	Managed Identity

3.2 Datasets

Source Dataset: ds_pRecord_csv

Reads Patient_Record.csv from the raw ADLS container. Uses comma as delimiter, backslash as escape character, and treats the first row as a column header. The schema contains 40 string-typed columns covering patient demographics, admission details, clinical measurements, and financial information.

Sink Dataset: ds_pRecord_parq

Writes Parquet files into the bronze folder of the patient-record ADLS container using Snappy compression. This Parquet output serves as the input for the MinorT Databricks notebook in the next pipeline step.

3.3 Schedule Trigger

Property	Value
Trigger Name	PatientData_Merge
Trigger Type	ScheduleTrigger
Frequency	Daily (interval: 1)
Time Zone	GMT Standard Time
State	Started (Active)

4. Databricks Transformation Notebooks

4.1 MinorT Notebook - Bronze to Silver

This notebook performs comprehensive data cleaning on the raw Parquet data from the Bronze layer using PySpark, then writes a clean, strongly-typed Delta table to the Silver layer under Unity Catalog.

4.1.1 Unity Catalog Setup

The notebook uses Databricks Unity Catalog for governed data access. An external volume is registered over the Bronze ADLS path, enabling schema-on-read access to Parquet files without copying data into managed storage. A dedicated schema is created to organise silver layer tables:

```
CREATE SCHEMA IF NOT EXISTS databricks_projects.silver
CREATE EXTERNAL VOLUME databricks_projects.patientrecord.my_volume
  LOCATION 'abfss://patient-record@<storage-account>.dfs.core.windows.net/bronze/';
```

4.1.2 Metadata Columns Added

Column	Source	Purpose
_ingestion_timestamp	current_timestamp()	Records when the row was processed
_source_file	_metadata.file_path	Full path of the source Parquet file
_source_file_name	_metadata.file_name	Filename of the source file
_file_modification_time	_metadata.file_modification_time	Last modified time of source file

4.1.3 Data Quality Rules Applied (Bronze to Silver)

The following 13+ cleaning transformations are applied in sequence:

Field(s)	Issue in Raw Data	Transformation Applied
patient_id	Mixed formats: plain integers, 'PT-19508' prefix, leading zeros	regexp_replace() strips 'PT-' and leading zeros; cast to Integer
patient_id (dedup)	Duplicate patient records present	Window function: row_number() partitioned by patient_id_clean, ordered by _file_modification_time DESC; keep rank 1 only
first_name / last_name	Mixed case ('SANDRA', 'anthony'), leading/trailing whitespace (' Andrew ')	trim() + initcap(); null substituted for empty strings and 'N/A' values
full_name	Not present in source data	Derived column: concat_ws(' ', first_name_clean, last_name_clean)
date_of_birth / admission_date / discharge_date	Multiple date formats: yyyy-MM-dd, MM/dd/yyyy, yyyy/MM/dd	coalesce() across three format-specific to_date() parsers; unmatched values become null
age	Sentinel values (-999), text suffix ('75 yrs'), negative numbers	Numeric extraction, range validation (0-130); computed from DOB and admission_date via months_between() when raw value is invalid
gender	Mixed representations: 'M', 'MALE', 'male', 'F', 'FEMALE', '0', '1'	Mapped to canonical 'Male' or 'Female'; unrecognised values become null

Field(s)	Issue in Raw Data	Transformation Applied
height_cm / weight_kg	Sentinel 'N/A', embedded units ('97.3 kg')	Sentinel null removal; regexp_replace() strips non-numeric characters; range validation applied
bmi	Missing, invalid, or out-of-range values	Computed from cleaned height and weight (weight_kg / (height_cm/100)^2); falls back to raw value if inputs unavailable
temperature_celsius	Unit string appended ('36.6 C')	Strip non-numeric characters; range validation (34.0 to 43.0 degrees C)
length_of_stay_days	Negative values where discharge date precedes admission date	Recomputed via datediff(discharge_date_clean, admission_date_clean); set to null when negative
discharge_date	Logically impossible dates (before admission date)	Set to null when computed length-of-stay is negative
email	Uppercase addresses, malformed emails (missing @ symbol)	lower() + trim() applied; regex validation against standard email pattern; null if invalid
insurance_provider	Mixed case: 'CENTENE', 'BlueCross BlueShield', 'KAISER PERMANENTE'	initcap() normalisation; null substituted for 'UNKNOWN' and 'N/A' values
insurance_claim_amount_usd	Negative monetary values present	Values below zero replaced with null
smoker	Mixed representations: 'Y', 'yes', '1', 'N', 'no', '0', 'Unknown'	Mapped to canonical 'Yes', 'No', or 'Former'; unrecognised values become null
blood_glucose_mgdl / creatinine_mgdl / hba1c_pct / heart_rate_bpm / oxygen_saturation_pct	Sentinel -999 values and physiologically impossible readings	Sentinel removal; range validation applied per field (e.g. glucose 20-700 mg/dL, O2 sat 50-100%)

4.1.4 Silver Table Output

The cleaned Silver dataset is written as a managed Delta table: databricks_projects.silver.patient_records_clean

Write Property	Value
Format	Delta Lake
Mode	overwrite (with overwriteSchema: true)
Table type	Managed table under Unity Catalog
Row count (verified)	952 unique patient records
Audit column added	_silver_timestamp (current_timestamp)

4.2 MajorT Notebook - Silver to Gold and Serving

This notebook reads the clean Silver Delta table and performs two operations: a Delta MERGE into the Gold layer for upsert semantics, and a JDBC write to Azure SQL Database for Power BI consumption.

4.2.1 Delta MERGE Logic (Gold Layer)

A Delta MERGE operation runs against the Gold table (databricks_projects.gold.patient_records_final) using patient_id as the merge key. The logic is:

- WHEN MATCHED AND target._silver_timestamp < source._silver_timestamp: update key clinical and demographic fields
- WHEN NOT MATCHED: insert all columns for new patients
- Result: fully idempotent upsert - re-running the pipeline produces the same result

Fields updated on a match: first_name, last_name, full_name, date_of_birth, age, gender, email, address, city, state, admission_date, discharge_date, length_of_stay_days, department, primary_diagnosis, _silver_timestamp.

4.2.2 Secret Management for JDBC

All database credentials are retrieved at runtime from a Databricks secret scope using dbutils.secrets.get(). No credentials appear in notebook code or version control:

Secret Key	Used For
ServerName	Azure SQL Server hostname
DBName	Target database name
UserName	SQL authentication username
AzSQLPass	SQL authentication password

4.2.3 JDBC Write Configuration

JDBC Property	Value / Setting
Driver	com.microsoft.sqlserver.jdbc.SQLServerDriver
Encryption	Enabled (encrypt=true, trustServerCertificate=false)
Batch size	10,000 rows per batch
Write mode	overwrite
Target table	dbo.patient_records_stg
Table lock	Enabled for write performance

5. Data Schema

5.1 Source Schema - 40 Columns

The raw CSV file contains 40 columns. All fields arrive as string type and require type casting and validation:

Column(s)	Category	Notes
patient_id	Identity	Mixed formats - requires normalisation
first_name, last_name	Demographics	Mixed case, whitespace issues
date_of_birth	Demographics	Multiple date formats present
age	Demographics	Sentinel values and text suffixes
gender	Demographics	Abbreviated and full word formats
ssn	PII	Partially masked in source
phone_number	Contact	Variable format
email	Contact	May be uppercase or malformed
address, city, state, zip_code	Location	Generally clean
admission_date, discharge_date	Admission	Multiple date formats
length_of_stay_days	Admission	May be negative (data error)
admission_type	Admission	Emergency/Urgent/Elective/Routine/Transfer
department, attending_physician	Clinical	Generally clean string fields
primary_diagnosis, secondary_diagnosis	Clinical	ICD diagnosis descriptions
medication_1, medication_2	Clinical	Drug names
height_cm, weight_kg, bmi	Biometrics	May include units or N/A
blood_pressure	Vitals	Systolic/Diastolic string (e.g. 167/61)
heart_rate_bpm, temperature_celsius, oxygen_saturation_pct	Vitals	Sentinel values possible
blood_glucose_mgdl, creatinine_mgdl, hba1c_pct	Lab values	-999 sentinel nulls present
insurance_provider	Financial	Mixed case
insurance_claim_amount_usd	Financial	Negative values possible
icu_days, readmission_30day_flag	Clinical	Integer flags
smoker, alcohol_use	Lifestyle	Multiple representations
discharge_disposition	Outcome	Discharge outcome category

5.2 Silver Schema - 37 Columns

After transformation, the Silver table contains 37 strongly-typed columns. Every raw field is replaced by a validated, cast counterpart. Three metadata columns are retained from ingestion and one audit timestamp is added:

Column Group	Key Columns	Clean Type
Patient Identity	patient_id, first_name, last_name, full_name	Integer / String
Demographics	date_of_birth, age, gender	Date / Long / String
Contact	email, address, city, state	String
Admission	admission_date, discharge_date, length_of_stay_days, admission_type, department	Date / Integer / String
Clinical	height_cm, weight_kg, bmi, heart_rate_bpm, temperature_celsius, oxygen_saturation_pct, blood_glucose_mgdl, creatinine_mgdl, hba1c_pct	Double
Financial	insurance_provider, insurance_claim_amount_usd, icu_days, readmission_30day_flag	String / Double / Integer
Lifestyle	smoker, alcohol_use	String
Audit	_silver_timestamp	Timestamp

6. Security and Governance

6.1 Secret Management with Azure Key Vault

All sensitive credentials are stored in Azure Key Vault and referenced at runtime. No credentials are hardcoded in pipeline JSON definitions, Databricks notebooks, or source code committed to version control.

Secret	Stored In	Consumed By
Databricks Personal Access Token	Azure Key Vault	ADF Databricks linked service for notebook execution
SQL Server hostname	Databricks secret scope	MajorT notebook JDBC connection
SQL Database name	Databricks secret scope	MajorT notebook JDBC connection
SQL username	Databricks secret scope	MajorT notebook SQL authentication
SQL password	Databricks secret scope	MajorT notebook SQL authentication

6.2 Unity Catalog Data Governance

Databricks Unity Catalog governs all Silver and Gold data assets. The catalog contains two schemas:

- silver schema - contains the managed Delta table: patient_records_clean
- gold schema - contains the managed Delta table: patient_records_final

External volumes and tables are registered through Unity Catalog using an Access Connector and external location credentials, ensuring full data governance, access auditing, and fine-grained access control over data stored in ADLS Gen2.

6.3 Network Security

- ADLS linked services use ADF-managed encrypted credentials
- JDBC connection to Azure SQL enforces TLS encryption (encrypt=true)
- SSL hostname validation enforced against Azure SQL certificate
- No sensitive resource names or endpoints appear in committed code

7. Repository Structure

The repository mirrors the pipeline layers and Azure resource types for easy navigation:

```
PatientRecordPipeline01/
|-- architecture/
|   +-- PatientRecord.png           # Architecture diagram
|-- dataset/
|   |-- ds_pRecord_csv.json         # ADF source dataset (CSV)
|   +-- ds_pRecord_parq.json       # ADF sink dataset (Parquet)
|-- linkedService/
|   |-- ls_pRecord_csv.json         # ADLS linked service (CSV source)
|   |-- ls_pRecord_parq.json       # ADLS linked service (Parquet sink)
|   |-- ls_Minor.json              # Databricks linked service
|   +-- PatientKV.json             # Azure Key Vault linked service
|-- notebooks/
|   |-- MinorT.ipynb               # Bronze to Silver transformation
|   +-- MajorT.ipynb              # Silver to Gold MERGE + SQL load
|-- pipeline/
|   +-- PatientRecordPipeline01.json # ADF master pipeline definition
|-- trigger/
|   +-- PatientData_Merge.json      # Daily schedule trigger
|-- sample-data/
|   +-- Patient_Record.csv          # Sample source data (approx 1,000 rows)
+-- Report/
    +-- PatientRecord.pbix          # Power BI report file
```

8. Setup and Deployment Guide

8.1 Prerequisites

- Azure subscription with Contributor access
- Azure Data Lake Storage Gen2 account with two containers created: raw and patient-record
- Azure Databricks workspace with Unity Catalog enabled and an Access Connector for ADLS
- Azure Key Vault with Databricks PAT token stored; a Databricks secret scope backed by Key Vault
- Azure SQL Database and a target schema (dbo) for the serving layer
- Power BI Desktop installed locally for report editing

8.2 Deployment Steps

- Step 1: Upload Patient_Record.csv to the raw container of your ADLS Gen2 account
- Step 2: Import the linked service, dataset, pipeline, and trigger JSON files into your Azure Data Factory. Update ADLS account names in linked service definitions to match your environment
- Step 3: Store the Databricks PAT token in Azure Key Vault. Create a Databricks secret scope backed by Key Vault and add SQL Server credentials (ServerName, DBName, UserName, AzSQLPass)
- Step 4: Upload MinorT.ipynb and MajorT.ipynb to your Databricks workspace. Update the notebookPath values in the pipeline JSON to reflect your workspace user path
- Step 5: In Databricks, create the Unity Catalog schemas: CREATE SCHEMA IF NOT EXISTS databricks_projects.silver and CREATE SCHEMA IF NOT EXISTS databricks_projects.gold
- Step 6: Register the external volume in Unity Catalog pointing to your ADLS bronze path
- Step 7: Create the Gold table shell: CREATE OR REPLACE TABLE gold.patient_records_final USING DELTA AS SELECT * FROM silver.patient_records_clean LIMIT 0
- Step 8: Activate the PatientData_Merge trigger in ADF, or trigger the pipeline manually to validate the full end-to-end flow
- Step 9: Open PatientRecord.pbix in Power BI Desktop, update the Azure SQL connection to your server and database, then publish to Power BI Service for shared access

9. Technical Skills Demonstrated

This project showcases the following competencies relevant to mid-level and senior Azure Data Engineer roles:

Skill Area	Demonstrated By
Azure Data Factory	Pipeline design with Copy Activity, Databricks Notebook activity, sequential dependency chaining, and Schedule Trigger configuration
Azure Data Lake Storage Gen2	Medallion architecture implementation across raw, bronze, silver, and gold containers with Parquet and Delta formats
PySpark / Azure Databricks	Complex data cleaning using Window functions, regexp_replace, multi-format date parsing, sentinel null handling, derived field computation, and schema selection
Delta Lake	Managed Delta tables, MERGE (upsert) operations with timestamp-based conditional updates, overwriteSchema for schema evolution
Databricks Unity Catalog	External volumes, managed schemas, external location credentials, and governed table registration across catalog hierarchy
Azure Key Vault	Secret integration in ADF (linked service via Key Vault reference) and Databricks (secret scope via dbutils.secrets.get)
JDBC / Azure SQL	Secure JDBC connection with batch write optimisation, credential scoping from secret scope, and serving layer loading
Data Quality Engineering	17+ field-level cleaning rules, Window-based deduplication, physiological range validation, multi-format parsing, and audit column tracking
Power BI	End-to-end report development connected to Azure SQL serving layer
Data Governance	Unity Catalog schema management, lineage tracking via metadata columns, and strict separation of concerns across medallion layers

10. Sample Data Quality Issues and Resolutions

The following examples illustrate the kinds of real-world data quality problems handled by this pipeline:

Field	Raw Value	Issue	Cleaned Value
patient_id	PT-19508	Non-numeric prefix	19508
patient_id	00034824	Leading zeros	34824
age	-999	Sentinel null value	Computed from date_of_birth
age	75 yrs	Text suffix appended	75
gender	male	Lowercase full word	Male
gender	0	Binary numeric code	Female
first_name	Andrew	Leading and trailing whitespace	Andrew
first_name	SANDRA	All uppercase	Sandra
temperature_celsius	36.6 C	Unit string appended	36.6
weight_kg	97.3 kg	Unit string embedded in value	97.3
email	SANDRA.T@AOL.COM	Uppercase email address	sandra.t@aol.com
email	thomas90aol.com	Missing @ symbol	null
length_of_stay_days	-3	Discharge before admission	null
discharge_date	2020-07-24	Before admission 2020-07-27	null
insurance_claim_amount_usd	-684.78	Negative monetary value	null
smoker	Y	Abbreviated flag	Yes
smoker	1	Binary numeric flag	Yes
blood_glucose_mgdl	-999	Sentinel null value	null
insurance_provider	KAISER PERMANENTE	All uppercase	Kaiser Permanente
date_of_birth	09/23/1995	MM/dd/yyyy format	1995-09-23 (Date type)